

SOFTWARE REQUIREMENTS SPECIFICATION

IT Helpdesk Ticketing System

Prepared for the Free State Department of Social Development
Central University of Technology, Free State — Diploma in Information Technology

Document Version: 2.0 (Combined Unit 1 & Unit 2 Update)

Date: August 2026

Technology Stack: Flutter · ASP.NET Core · Supabase

Table of Contents

1. Project Introduction
2. Stakeholder Profile and Current Process
3. Problem Statement
4. Project Aim and Objectives
5. Project Scope
6. Stakeholder Needs
7. Functional Requirements
8. Non-Functional Requirements
9. Integration Requirements
10. Requirements Change Log
11. Refined Project Scope

1. Project Introduction

The IT Helpdesk Ticketing System is a software solution proposed to replace the informal, ad-hoc method currently used to report and resolve information technology (IT) issues within the Free State Department of Social Development (DSD). The Department relies on a decentralised network of IT support staff who assist end users across multiple directorates and service points. At present there is no formal, trackable channel for logging, assigning, or resolving IT-related issues raised by departmental staff.

This project was identified through direct observation of, and consultation with, IT support staff and general departmental users during an academic workplace-based engagement at the Department. The observed gap — the absence of a structured logging and tracking mechanism for IT support requests — forms the basis for this proposed system.

The proposed solution consists of three integrated components: a Flutter-based mobile application through which staff can log and track support tickets from any device; an ASP.NET Core Web API that implements the system's business logic and serves both the mobile and web clients; and a Supabase-hosted PostgreSQL database that provides persistent storage, authentication and real-time data synchronisation. Together these components form a single, cohesive helpdesk platform.

This document is a combined and updated Software Requirements Specification (SRS), consolidating and refining the requirements gathered during the initial feasibility investigation (Unit 1) and the subsequent risk and design analysis (Unit 2) into a single authoritative reference for system design and development.

2. Stakeholder Profile and Current Process

2.1 Organisation

The Free State Department of Social Development is a provincial government department responsible for delivering social welfare, community development and social relief services across the Free State province. The Department operates from a head office and several regional and satellite offices, each equipped with computers, printers, networking equipment and departmental software systems that require ongoing IT support.

2.2 Stakeholders and Users

Stakeholder / User Group	Role in the System	Interest / Concern
IT Support Staff / Technicians	Receive, triage, and resolve logged tickets	Need a clear, prioritised queue of outstanding issues and a record of work performed
General Departmental Staff (End Users)	Log IT issues and track resolution progress	Need a simple way to report problems without relying on informal channels, and visibility into progress
IT Manager / Supervisor	Oversees ticket volumes, staff workload and resolution times	Needs reporting and dashboard visibility to manage the IT support function and demonstrate service delivery
Departmental Management	Indirect stakeholder; relies on IT systems functioning reliably	Needs assurance that service delivery is not disrupted by unresolved IT issues
Development Team (Student Project Group)	Designs, builds and maintains the system	Needs clear, testable requirements to guide development

2.3 Current Process

Currently, when a staff member experiences an IT issue — such as a malfunctioning computer, printer fault, network outage, software error or account access problem — they report it informally, most commonly via WhatsApp messages, phone calls, or by physically walking to an IT technician's desk. There is no centralised system for capturing these requests.

2.4 Identified Problems with the Current Process

- Issues reported verbally or via chat messages are frequently lost, forgotten, or duplicated because there is no single record of what has been reported.
- There is no way to track whether an issue has been assigned, is in progress, or has been resolved.
- IT technicians have no prioritised queue and often work reactively, addressing whoever contacts them most recently or most persistently.
- There is no historical record of recurring issues, making it difficult to identify patterns (e.g. a printer that fails repeatedly) or to plan proactive maintenance.
- Management has no visibility into IT support workload, response times, or resolution times, making it difficult to justify staffing or budget decisions.
- Accountability is weak: it is often unclear who was responsible for resolving a given issue, or whether it was resolved at all.

3. Problem Statement

The Free State Department of Social Development currently has no formal system for logging, tracking, and managing internal IT support requests. Staff members experiencing technical issues must rely on informal, undocumented communication channels such as WhatsApp messages or in-person requests to reach IT support staff.

This lack of structure means that reported issues are frequently lost or forgotten, response and resolution times cannot be measured, and there is no accountability for outstanding work. The affected users include general departmental staff, who experience delays and uncertainty when reporting problems, and IT support staff, who lack the tools to manage their workload effectively.

The consequences of this situation include reduced staff productivity while IT issues remain unresolved, duplicated effort when multiple people report or attempt to fix the same issue independently, an inability to identify recurring technical problems, and a lack of management data to support resourcing or process-improvement decisions.

There is a clear need for a structured, digital ticketing system that allows staff to log issues consistently, allows technicians to manage and prioritise their workload, and provides management with visibility into the performance of the IT support function.

4. Project Aim and Objectives

4.1 Project Aim

The aim of this project is to design and develop an integrated IT Helpdesk Ticketing System — comprising a Flutter mobile application and an ASP.NET Core web-based administrative platform, backed by a Supabase database — that enables Free State Department of Social Development staff to log, track and resolve IT support requests in a structured, accountable and measurable way.

4.2 Project Objectives

1. Enable staff to log IT support tickets from a mobile device, capturing issue category, description, priority and, optionally, a photo attachment.
2. Provide IT technicians with a web-based dashboard to view, assign, prioritise, and update the status of tickets.
3. Implement a notification mechanism that informs users when their ticket status changes.
4. Maintain a complete, searchable historical record of all logged tickets and their resolution outcomes.
5. Provide management-level reporting on ticket volumes, average resolution time, and technician workload.
6. Implement secure, role-based authentication so that staff, technicians and administrators only access functionality relevant to their role.
7. Ensure the system architecture integrates Flutter, ASP.NET Core and Supabase into a single, reliable data-consistent solution.

5. Project Scope

5.1 Deliverables

- A Flutter mobile application for logging and tracking IT support tickets (Android, with iOS-compatible codebase).
- An ASP.NET Core Web API implementing authentication, ticket management, assignment, and reporting logic.
- A web-based administrative dashboard (for IT technicians and the IT manager) for ticket triage, assignment and reporting.
- A Supabase-hosted PostgreSQL database with defined schema, authentication configuration, and storage buckets for attachments.
- Supporting project documentation: this SRS, system design documentation, and a basic user guide.

5.2 Acceptance Criteria

- A staff member can register/log in, submit a new ticket with a category and description, and view its status.
- An IT technician can log in, view a list of open tickets, assign a ticket to themselves or a colleague, and change its status through defined stages (Open → In Progress → Resolved → Closed).
- The system correctly persists all ticket data in the Supabase database and reflects updates across both the mobile app and web dashboard.
- The IT manager can generate a basic report showing ticket counts by status and average resolution time over a selected period.
- Role-based access control prevents general staff from accessing technician or administrative functionality.

5.3 Exclusions

- Integration with external, Department-wide enterprise systems (e.g. existing HR or asset-management systems) is out of scope for this iteration.
- Automated ticket routing using artificial intelligence or machine learning is out of scope.

- A public-facing iOS App Store or Google Play Store release is out of scope; the mobile application will be distributed as an installable build for demonstration and evaluation purposes.
- Offline-first mobile functionality (full local data capture and sync without any connectivity) is out of scope; basic connectivity is assumed.

5.4 Constraints

- The project must be completed within the academic timeline set for the module.
- Development is carried out by a student project team with limited prior production deployment experience.
- The system relies on the free/low-cost tiers of Supabase, which impose limits on database size, bandwidth and concurrent connections.
- Access to real Departmental data and infrastructure is limited; development and testing will primarily use representative sample data.

5.5 Assumptions

- Departmental staff have access to a smartphone or desktop computer with internet connectivity sufficient to use the application.
- The Department will designate at least one IT technician account and one IT manager account for evaluation and testing.
- Supabase remains available as a hosting platform for the duration of the project.

6. Stakeholder Needs

Stakeholder needs were gathered through direct observation of the current informal reporting process and informal consultation with IT support staff and general users at the Department. The following needs were confirmed as most significant:

Stakeholder	Confirmed Need
General staff (end users)	A simple, quick way to log an IT issue without needing to know who to contact, and the ability to check progress without following up manually
IT support staff	A single, prioritised view of all open issues, with the ability to claim, update and close tickets, replacing scattered chat messages
IT manager	Visibility into ticket volumes, outstanding workload, and resolution turnaround time to support planning and reporting
All users	Confidence that a logged issue will not be lost or forgotten, and a record they can refer back to

7. Functional Requirements

The functional requirements below are grouped by capability and each is traced to the stakeholder need it addresses.

ID	Requirement	Linked Need
FR-01	Users shall be able to register and log in using a secure account (staff, technician, or manager role).	Confidence issues are not lost; role-based access
FR-02	Staff shall be able to create a new ticket by specifying a category, description,	Simple, quick way to log

ID	Requirement	Linked Need
	priority, and optional photo attachment.	an issue
FR-03	The system shall automatically timestamp and assign a unique reference number to each ticket on creation.	Confidence a ticket is recorded
FR-04	Staff shall be able to view a list of their own submitted tickets and the current status of each.	Visibility into progress
FR-05	IT technicians shall be able to view a list of all open/unassigned tickets, filterable by category and priority.	Prioritised view of open issues
FR-06	IT technicians shall be able to assign a ticket to themselves or reassign it to another technician.	Replacing scattered chat messages
FR-07	IT technicians shall be able to update ticket status through defined stages: Open, In Progress, Resolved, Closed.	Single view + accountability
FR-08	IT technicians shall be able to add resolution notes/comments to a ticket.	Historical record
FR-09	The system shall notify a staff member when the status of their ticket changes.	Visibility without manual follow-up
FR-10	The IT manager shall be able to view a dashboard summarising ticket counts by status, category, and technician.	Reporting for planning
FR-11	The IT manager shall be able to generate a report of average resolution time over a selected date range.	Turnaround-time visibility
FR-12	The system shall maintain a searchable log of all closed tickets for historical reference.	Identifying recurring issues
FR-13	The system shall restrict access to technician and manager functionality based on the authenticated user's role.	Access control

8. Non-Functional Requirements

Category	Requirement
Usability	The mobile application shall allow a first-time user to log a new ticket in no more than three screens/steps, without requiring training.
Usability	The web dashboard shall present open tickets in a clearly prioritised, sortable list understandable to non-technical IT manager staff.
Performance	Ticket list views shall load within 3 seconds under normal network conditions with up to 500 stored tickets.
Performance	Status updates made by a technician shall be reflected to the relevant staff member's view within 10 seconds.
Security	All user passwords shall be stored using Supabase's built-in hashed authentication; passwords shall never be stored or transmitted in plain text.
Security	All communication between the Flutter app, the ASP.NET Core API, and Supabase shall be encrypted using HTTPS/TLS.
Security	The system shall enforce role-based authorisation on every API endpoint, not only in the client interface.
Security	The system shall comply with the principles of South Africa's Protection of Personal Information Act (POPIA) regarding storage of staff personal data.

Category	Requirement
Availability	The system shall target 95% uptime during departmental working hours (08:00–16:30, weekdays), subject to Supabase platform availability.
Reliability	No ticket data shall be lost once successfully submitted; all writes shall be confirmed before the user is shown a success message.
Maintainability	The ASP.NET Core API shall be structured using a layered architecture (controllers, services, data access) to support future extension.
Scalability	The database schema shall support growth to at least 10,000 tickets without requiring structural redesign.
Compatibility	The Flutter application shall run on Android 9.0 and above; the web dashboard shall support current versions of Chrome and Edge.

9. Integration Requirements

The system is composed of three distinct technologies that must operate together as a single, consistent solution: Flutter (mobile client), ASP.NET Core (application/business logic layer), and Supabase (database, authentication, and storage). The integration approach is defined as follows:

9.1 Overall Architecture

The ASP.NET Core Web API acts as the central business-logic layer for the system. Both the Flutter mobile application and the web administrative dashboard communicate with this API over HTTPS using RESTful JSON endpoints; neither client accesses the database directly. The API, in turn, communicates with the Supabase-hosted PostgreSQL database using Entity Framework Core with the Npgsql provider, applying all business rules, validation, and role-based access control before any data reaches the database.

9.2 Authentication Integration

Supabase Auth is used as the single source of truth for user identity. Users authenticate through Supabase Auth (email/password), which issues a JSON Web Token (JWT). This token is passed by the Flutter app and the web dashboard on every request to the ASP.NET Core API, which validates the token's signature and claims before processing the request and applying role-based authorisation (staff, technician, or manager).

9.3 Data Flow

8. A staff member submits a ticket from the Flutter app, which sends an authenticated HTTPS request to the ASP.NET Core API.
9. The API validates the request, applies business rules (e.g. generating a reference number), and writes the ticket record to the Supabase PostgreSQL database via Entity Framework Core.
10. If a photo is attached, the Flutter app uploads it directly to a designated Supabase Storage bucket, and stores the resulting file reference with the ticket record via the API.
11. IT technicians interact with the same API from the web dashboard to retrieve, update, and assign tickets; all changes are persisted to the same underlying Supabase database, ensuring both clients always reflect a single consistent state.
12. Where near real-time updates are required (e.g. a technician picking up a ticket), the system uses Supabase's real-time change-subscription capability, allowing the web dashboard to refresh automatically without manual polling.

9.4 Integration Risks and Mitigations

Risk	Mitigation
JWT validation misconfiguration between Supabase Auth and ASP.NET Core could allow unauthorised access.	Use Supabase's published JWT signing secret/JWKS configuration in ASP.NET Core's authentication middleware, and test role-based access explicitly for each endpoint.
Direct client access to Supabase could bypass API business rules if Row Level Security (RLS) is not properly configured.	Enable and test Supabase Row Level Security policies as a defence-in-depth layer, even though clients are expected to route through the API.
Network interruptions on mobile could cause a ticket submission to appear to fail when it partially succeeded.	Implement idempotent ticket-creation requests and clear success/failure feedback in the Flutter app.

10. Requirements Change Log

The following changes were made to the requirements following the feasibility investigation (Unit 1) and subsequent risk analysis (Unit 2), together with the reasons for each change.

#	Change	Reason
1	Reduced initial scope from an all-in-one IT service management system (incorporating asset management) to a focused ticketing system.	Feasibility analysis showed combining ticketing with full asset management within the project timeline carried excessive risk; the ticketing gap was assessed as the more urgent and clearly evidenced need.
2	Selected Supabase (managed PostgreSQL) instead of a self-hosted SQL Server or local database for the shared backend.	Risk analysis identified self-hosted database administration and remote-access configuration as a significant risk for a student team with limited infrastructure experience; Supabase reduces this risk while still integrating cleanly with ASP.NET Core via Entity Framework Core.
3	Moved authentication responsibility to Supabase Auth rather than building custom authentication in ASP.NET Core.	Reduces security risk associated with building and maintaining custom password storage and token issuance, and shortens development time.
4	Removed AI-based automatic ticket categorisation from scope.	Assessed during risk analysis as high effort relative to benefit within the available timeline; manual category selection meets the core stakeholder need at lower risk.
5	Deferred offline mobile functionality (local caching and sync) to a future phase.	Added significant technical complexity and testing burden; basic connectivity was confirmed as an acceptable assumption for the target users.
6	Added Row Level Security (RLS) policies on Supabase as an explicit non-functional/security requirement.	Identified during risk analysis as a necessary defence-in-depth measure given that Supabase is directly reachable by client applications.

11. Refined Project Scope

Following feasibility and risk analysis, the scope taken forward into system design is a single-purpose IT Helpdesk Ticketing System comprising:

- A Flutter mobile application enabling staff to register, log in, submit tickets with category/description/priority/optional photo, and track ticket status.
- A web-based technician/manager dashboard (served by the ASP.NET Core solution) for viewing, assigning, updating and reporting on tickets.
- An ASP.NET Core Web API providing all business logic, validation and role-based authorisation, integrated with Supabase for authentication, data storage and file storage.
- Basic reporting: ticket counts by status/category and average resolution time.

Explicitly excluded from this refined scope: asset/inventory management, AI-driven categorisation or routing, offline-first mobile functionality, and integration with external Departmental enterprise systems. These items are documented as candidates for future phases beyond the current academic project timeline.

This refined scope reflects a solution that is technically achievable within the project's constraints, directly addresses the confirmed stakeholder needs, and integrates Flutter, ASP.NET Core and Supabase as a single, coherent system ready to proceed to detailed system design.